

Introduction to anemoi-training

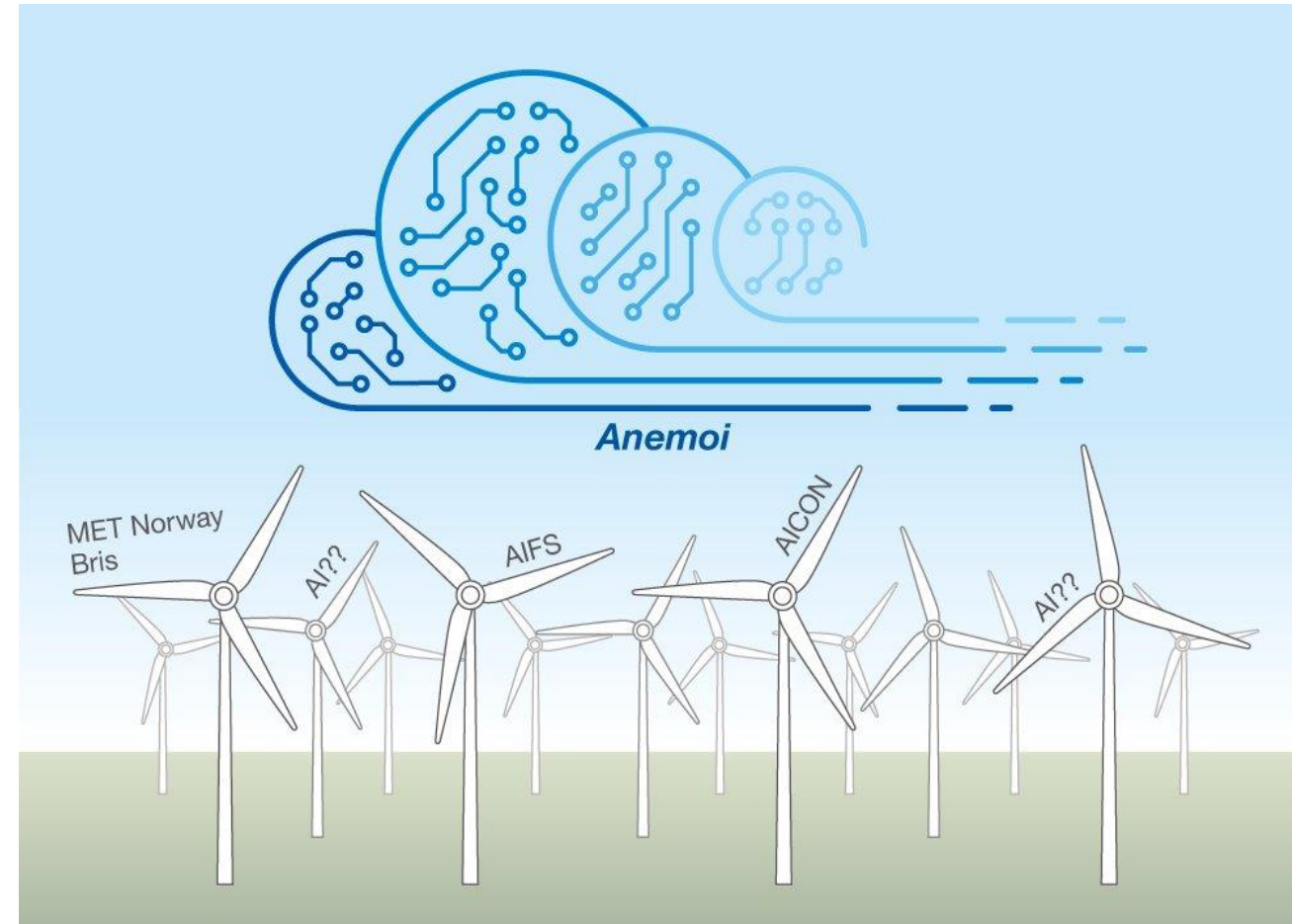
Vera Gahlen

On behalf of many people at ECMWF



Anemoi framework

- Develop machine learning (ML) weather forecasting models.
 - Dataset preparation tools.
 - Graph creation tools.
 - ML model training support.
 - Inference tools integrated with verification software.
 - Registry for datasets and trained models.
- **Benefits:**
 - Simplifies shared meteorological challenges.
 - Enables training of models using existing recipes and custom data.



Anemoi framework

- Develop machine learning (ML) weather forecasting models.
 - Dataset preparation tools.
 - Graph creation tools.
 - ML model training support.
 - Inference tools integrated with verification software.
 - Registry for datasets and trained models.
- **Benefits:**
 - Simplifies shared meteorological challenges.
 - Enables training of models using existing recipes and custom data.

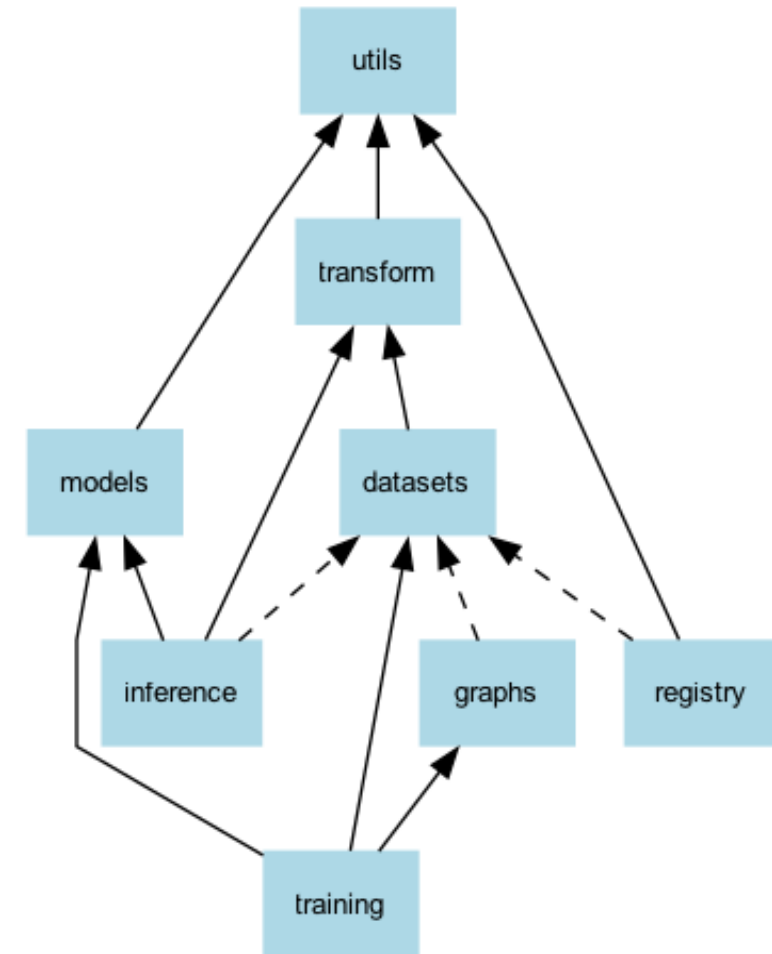
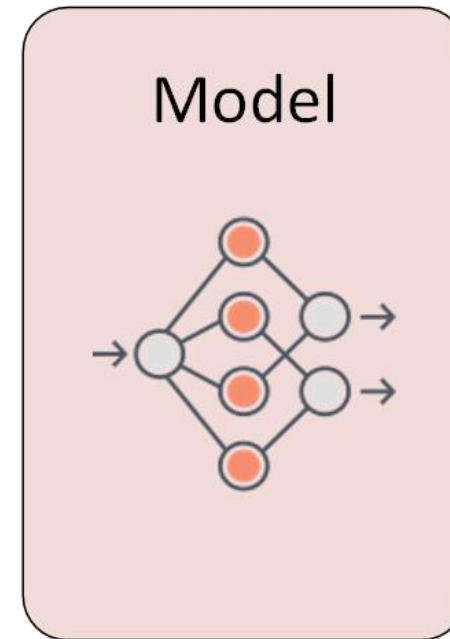
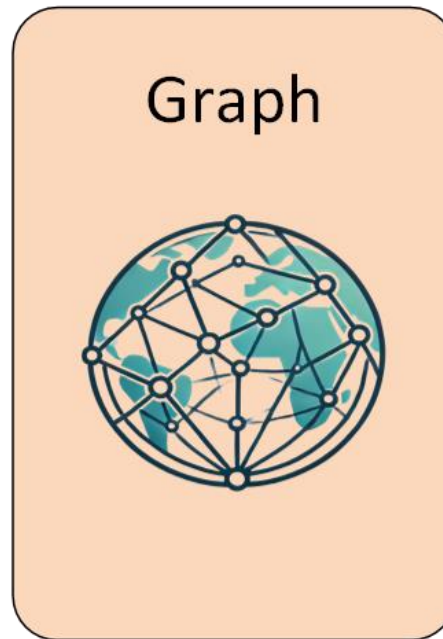
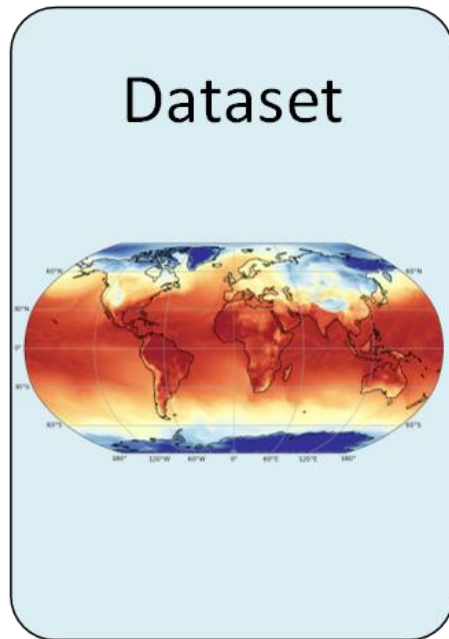


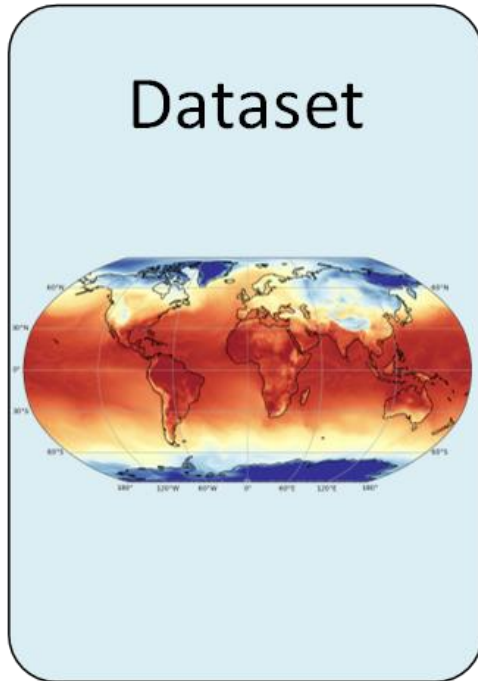
Diagram of dependencies within anemoi packages.

Getting started

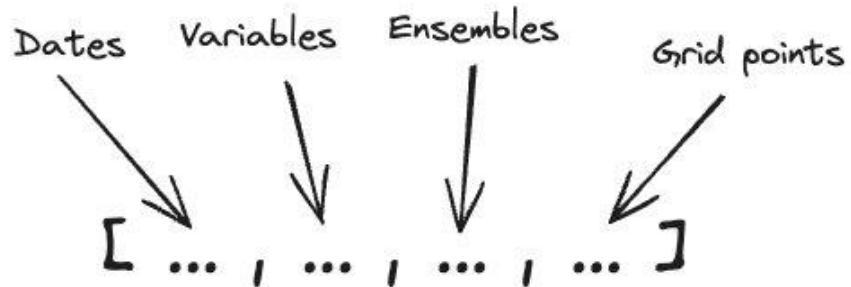
To train a data-driven weather forecasting model in **Anemoi**, three main components are needed:



anemoi-datasets

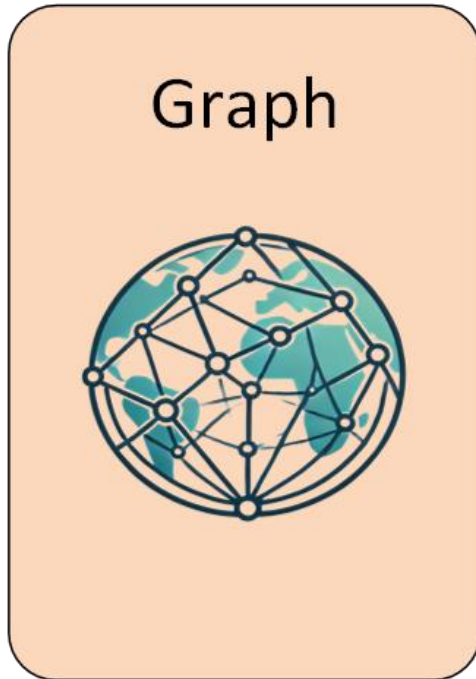


anemoi-datasets.readthedocs.io



- Create "machine-learning ready" datasets for training data-driven weather forecasts.
- Make the loading of data samples as efficient as possible
 - I/O operations are minimised
- Zarr
 - Offers an array-like view on chunks
 - Each file is a single date
- Using datasets
 - Subsetting datasets (time, variables, member, ...)
 - Combining datasets (join, concat, cutout, ensemble, ...)

anemoi-graphs



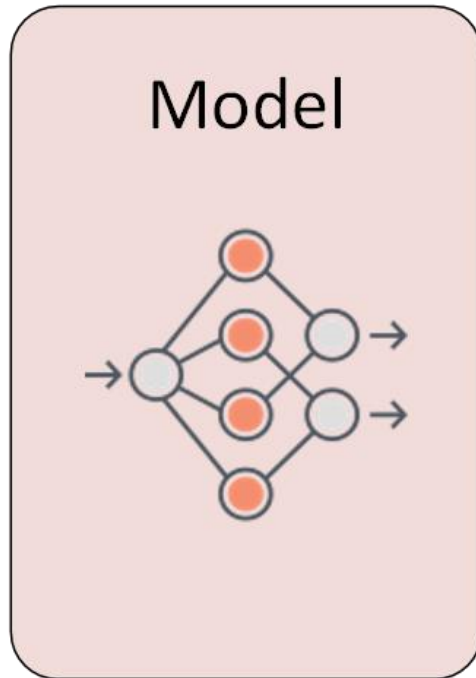
anemoi-graphs.readthedocs.io

In Anemoi, it is represented by a `torch_geometric.data.HeteroData` object, and stored in a .PT file.

- A graph is a structure composed of:
 - **Nodes:** Represent locations in space.
 - **Edges:** Define how information flows between nodes.

```
HeteroData(  
  data={  
    x=[40320, 2], # coordinates in radians (lat in [-pi/2, pi/2], lon in [0, 2pi])  
    node_type='ZarrDatasetNodes',  
    area_weight=[40320, 1],  
  },  
  hidden={  
    x=[10242, 2], # coordinates in radians (lat in [-pi/2, pi/2], lon in [0, 2pi])  
    node_type='TriNodes',  
    area_weight=[10242, 1],  
  },  
  (data, to, hidden)={  
    edge_index=[2, 62980],  
    edge_type='CutOffEdges',  
    edge_length=[62980, 1],  
    edge_dirs=[62980, 2],  
  },  
  (hidden, to, hidden)={  
    edge_index=[2, 81900],  
    edge_type='MultiScaleEdges',  
    edge_length=[81900, 1],  
    edge_dirs=[81900, 2],  
  },  
  (hidden, to, data)={  
    edge_index=[2, 120960],  
    edge_type='KNNEdges',  
    edge_length=[120960, 1],  
    edge_dirs=[120960, 2],  
  }  
)
```

Console log of a graph created with anemoi-graphs.



anemoi-models.readthedocs.io

- Graph Neural Networks (GNNs) are a type of neural network designed to operate on **graph-structured** data.
 - $GNN(G) = G'$, where G, G' are graphs
- Node features are update based on message passing:
 - We will look into this tomorrow
 - For now: the graph encodes the structure of our input/output data and how we want information to flow
- Based on the graph, the format of the data, and additional configurations, anemoi-models sets up a pytorch model which we can train

anemoi-training

- Code to train models, using torch-lightning and Hydra
 - Multi-node/multi-GPU training support
 - Deterministic and probabilistic training
 - Callbacks for profiling evaluating, plotting and logging intermediate results
 - Implement various losses, more can be easily added
 - Interfaces with trackers such as mlflow
- Highly configurable
- Interfaces with:
 - anemoi-dataset via data loaders
 - anemoi-models via Hydra configuration
 - anemoi-inference via metadata-rich checkpoints

anemoi-training

```
Model / GNN.yml
activation: GELU
num_channels: 512

model:
  _target_:
anemoi.models.models.encoder_processor_decoder.AnemoiModelEncProcDec

processor:
  _target_:
anemoi.models.layers.processor.GNNProcessor
  _convert_: all
  activation: ${model.activation}
  trainable_size:
${model.trainable_parameters.hidden2hidden}
  sub_graph_edge_attributes:
${model.attributes.edges}
  num_layers: 16
  num_chunks: 2
  mlp_extra_layers: 0

encoder:
  _target_:
anemoi.models.layers.mapper.GNNForwardMapper
```

anemoi-training train model=gnn

```
Model / GraphTransformer.yml
activation: GELU
num_channels: 1024

model:
  _target_:
anemoi.models.models.encoder_processor_decoder.AnemoiModelEncProcDec

processor:
  _target_:
anemoi.models.layers.processor.GraphTransformerProcessor
  _convert_: all
  activation: ${model.activation}
  trainable_size:
${model.trainable_parameters.hidden2hidden}
  sub_graph_edge_attributes:
${model.attributes.edges}
  num_layers: 16
  num_chunks: 2
  mlp_hidden_ratio: 4 # GraphTransformer
  num_heads: 16 # GraphTransformer

encoder:
```

anemoi-training train model=graphtransformer

```
Model / Transformer.yml
activation: GELU
num_channels: 1024

model:
  _target_:
anemoi.models.models.encoder_processor_decoder.AnemoiModelEncProcDec

processor:
  _target_:
anemoi.models.layers.processor.TransformerProcessor
  _convert_: all
  activation: ${model.activation}
  num_layers: 16
  num_chunks: 2
  mlp_hidden_ratio: 4 # Transformer only
  num_heads: 16 # Transformer only
  window_size: 512
  dropout_p: 0.0

encoder:
  _target_: anemoi.models.layers.mapper.GraphTransformerFor
```

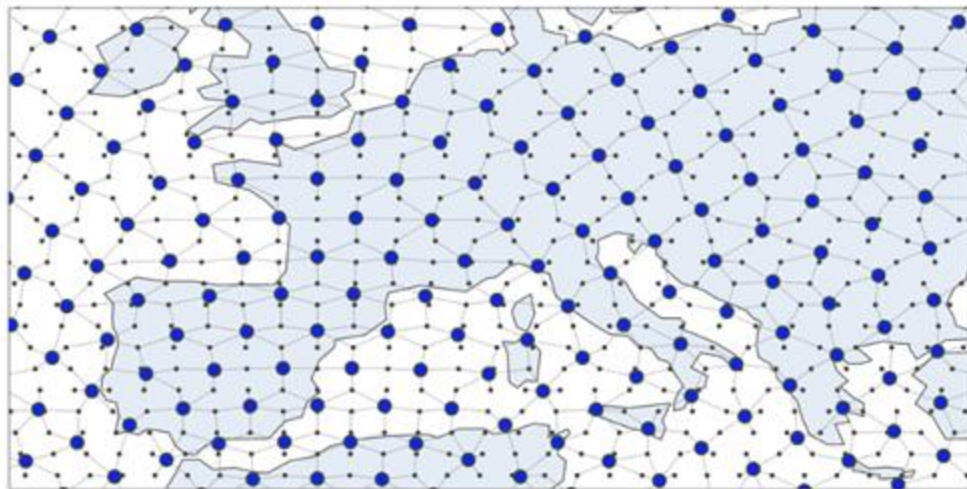
anemoi-training train model=transformer

- Make it easy to switch components
- Allow for reproduceable training
- Easy to extend with new models and components

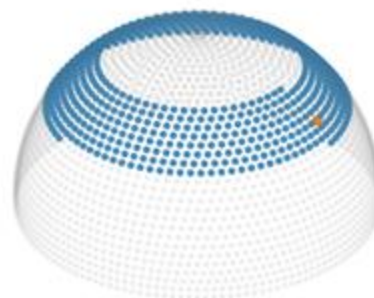
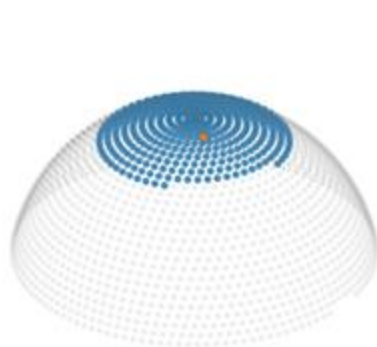
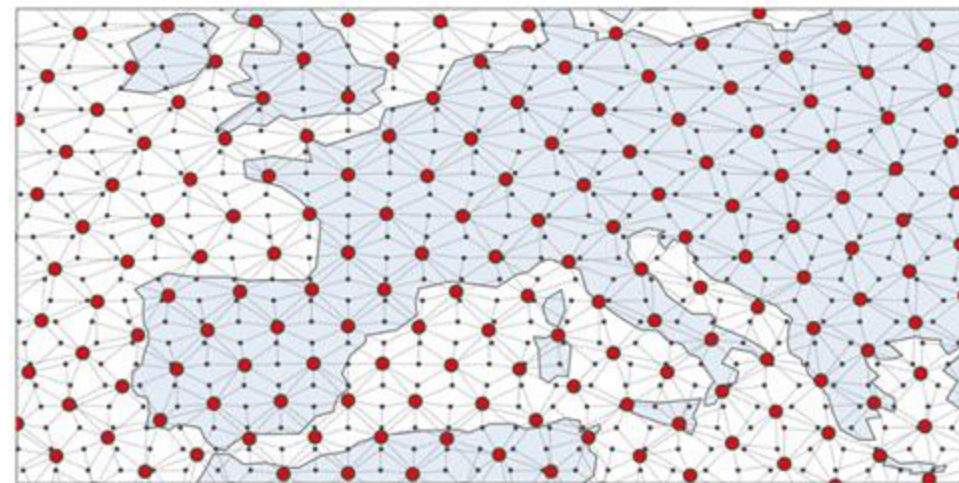
anemoi-training

Example: current AIFS configuration

Graph Encoder

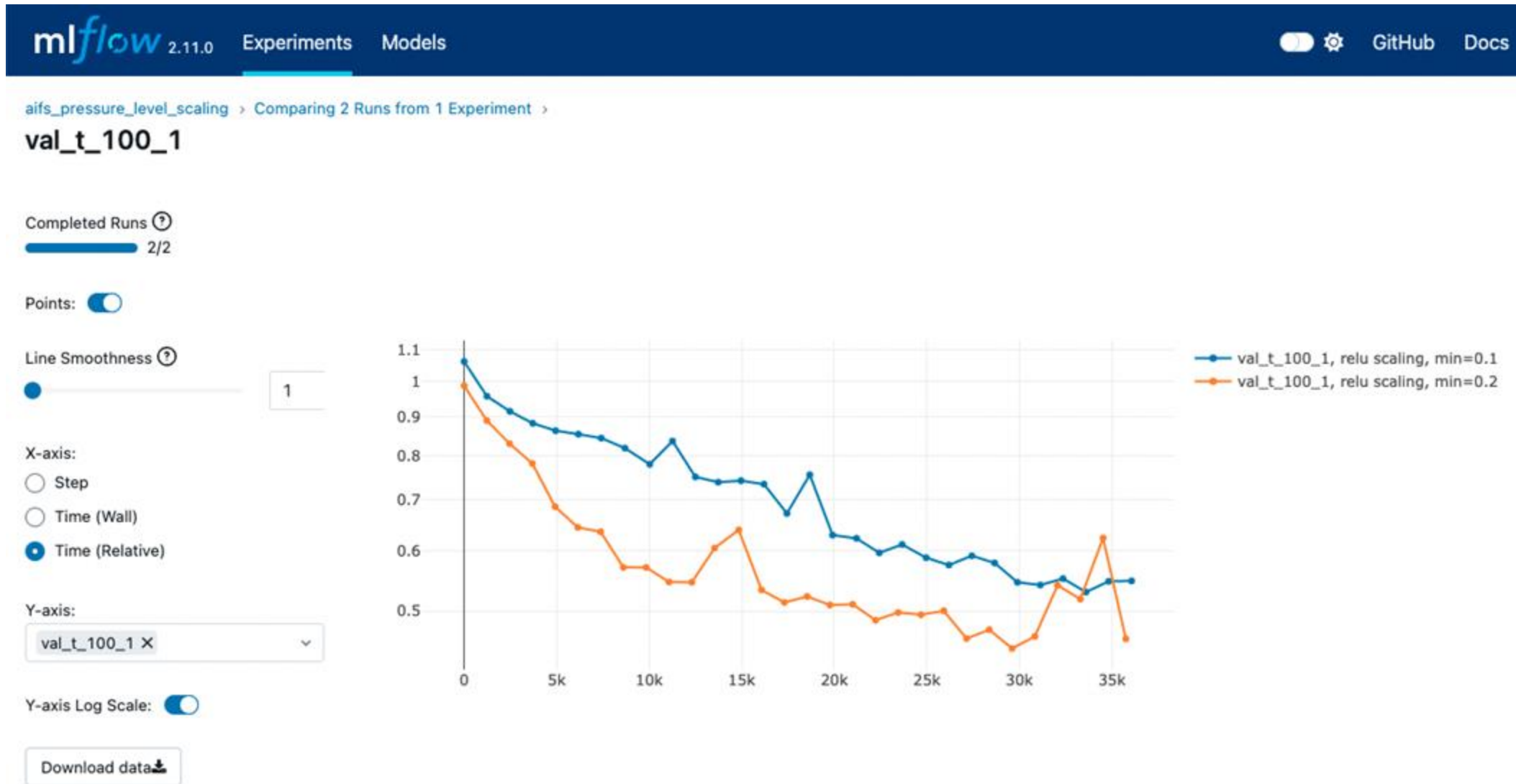


Graph Decoder



Transformer processor

anemoi-training – MLFlow servers for Experiment Tracking



Getting started with anemoi-training

Training Configuration

`anemoi-core / training / src / anemoi / training / config /`

Name
..
data
dataloader
datamodule
diagnostics
graph
hardware
model
training
__init__.py
config.yaml
debug.yaml
diffusion.yaml
ensemble_crps.yaml
hierarchical.yaml
interpolator.yaml
lam.yaml
stretched.yaml

```
1 defaults:
2 - data: zarr
3 - dataloader: native_grid
4 - diagnostics: evaluation
5 - datamodule: single
6 - hardware: example
7 - graph: multi_scale
8 - model: gnn
9 - training: default
10 - _self_
11
12
13 # set to true to switch on config validation
14 config_validation: True
```

```
1 num_channels: 512
2 cpu_offload: False
3
4 keep_batch_sharded: True
5
6 model:
7   _target_: anemoi.models.models.AnemoiModelEncProcDec
8
9 layer_kernels:
10   # The layer_kernels can be adjusted per model component, but are d
11   LayerNorm:
12     # The GNN requires the autocast layer norm, otherwise its memory
13     _target_: anemoi.models.layers.normalization.AutocastLayerNorm
14   Linear:
15     _target_: torch.nn.Linear
16   Activation:
17     _target_: torch.nn.GELU
18
19 processor:
20   _target_: anemoi.models.layers.processor.GNNProcessor
21   trainable_size: ${model.trainable_parameters.hidden2hidden}
22   sub_graph_edge_attributes: ${model.attributes.edges}
23   num_layers: 16
```

- Set up to enable modifying key components of the models and training without code changes
- Using the [Hydra](#) config system
- Templates for various use cases based on a hierarchy of yaml files

Command Line Interface

- Generate user config folder:

```
>>> anemoi-training config generate
```

- Start a training run with the *default configurations*:

```
>>> anemoi-training train
```

- Start a training run with user settings:

```
>>> anemoi-training train --config-name=my_config.yaml
```

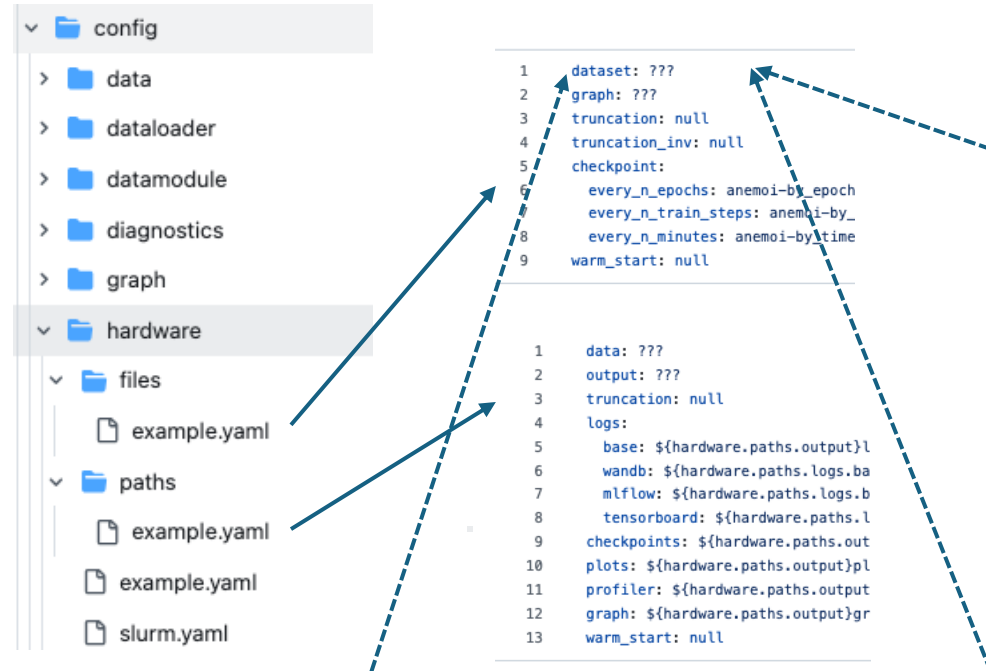
Three ways to modify config entries:

- Edit directly in the copied config folder
- Add overrides in the top-level config
- Use cli overrides

The training script will intentionally crash as it does not know where your data is stored.

These missing values in the configuration are placeholders for the user to fill in marked with **???**.

How to point to where your data is stored



Option 1 (in low-level config)

- Picked up by top-level configs
- Can edit here directly
- Sets a default for all top-level configs pointing to this file

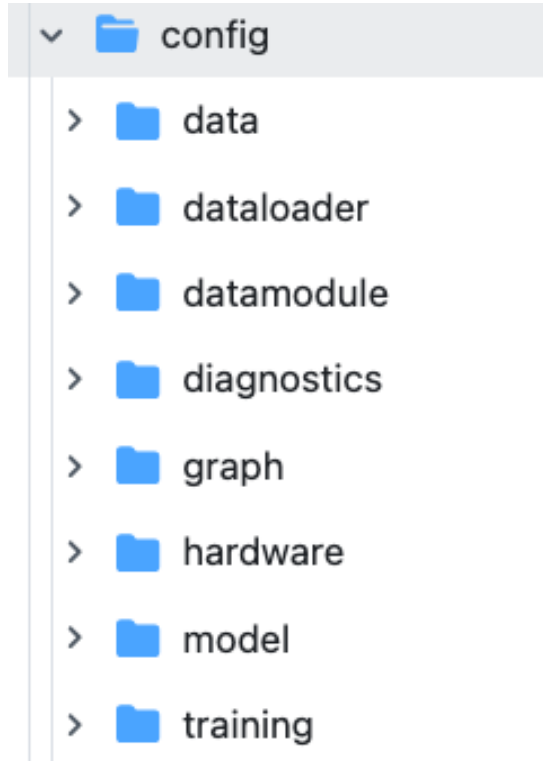
Option 3 (as cli override)

```
>>> anemoi-training train --config-name=config.yaml
hardware.files.dataset="my_dataset_name.zarr"
```

Option 2 (add overrides in top-level config)

```
1 # Modifications for the basic template "config.yaml"
2 hardware:
3   files:
4     dataset: anemoi-integration-tests/training/datasets/aifs-ea-an-oper-0001-mars-o48-1979-19-6h-v6-testset.zarr
5   paths:
6     data: https://object-store.os-api.cci1.ecmwf.int/ml-tests/test-data/samples
7
```

Configuration Overview



- Data: prognostic/diagnostic/forcing variables, normalizers
- Dataloader: batch_size, training.end, validation.start
- Diagnostics: callbacks=[], log
- Graph
- Hardware: files.dataset, paths.data
- Model: num_channels (defaults: gnn.yaml, graphtransformer.yaml)
- Training

Useful links

- Docs: <https://anemoi.readthedocs.io/projects/training/en/latest/>
- Config templates: <https://github.com/ecmwf/anemoi-core/tree/main/training/src/anemoi/training/config>
- Integration tests configs: <https://github.com/ecmwf/anemoi-core/tree/main/training/tests/integration>

Demo

Collaboration, without aggregation around a single model

